

Informe de Revision del Sistema AVA

Generado: 19/06/2026 01:45. Alcance: revision tecnica, flujo funcional, pruebas ejecutadas, riesgos y fallas detectadas. Se omitio intencionalmente el modulo indicado por el usuario.

Resumen ejecutivo. El sistema corre sobre Laravel 8.83.29 con PHP 8.3.30 y tiene una estructura funcional clara, pero presenta riesgos importantes: dependencias vulnerables, base de datos desalineada por migraciones pendientes, seeders rotos, rutas publicas de prueba/utilidad, pruebas automatizadas casi inexistentes y configuracion local con `APP_DEBUG=true` . La app puede responder en escenarios simples, pero no hay suficiente cobertura para asegurar operaciones criticas.

Flujo General del Sistema

- Usuarios sin sesion llegan a `/` o `/login` , donde se muestra la vista de ingreso `resources/views/auth/login.blade.php` .
- El login usa `LoginRequest` , valida usuario/password, aplica rate limit de 5 intentos y regenera la sesion al autenticar.
- Despues del login, los roles `master` y `admin` son enviados a `/main` ; el rol `worker` va a `/sales` .
- El sistema guarda una bandera de sesion para solicitar PIN/colaborador despues de entrar.
- Los modulos de trabajo estan en un grupo con middleware `auth` : clientes, productos, proveedores, ventas, compras, pagos, creditos/contratos, almacen, transferencias, cierres, usuarios, tanques, islas, surtidores, boveda, mediciones y reportes.
- La separacion por sede aparece en muchos controladores mediante `auth()->user()->location_id` ; esto ayuda a filtrar datos, pero la autorizacion esta repartida en codigo en vez de estar centralizada en policies/middleware de rol.

Pruebas Ejecutadas

Prueba	Resultado	Observacion
<code>php -v</code>	PHP 8.3.30	Todos los comandos PHP muestran warning por <code>pdo_oci</code> no disponible.
<code>php artisan --version</code>	Laravel 8.83.29	Version antigua; conviene planificar actualizacion.
<code>composer validate --strict</code>	Valido	El <code>composer.json</code> es sintacticamente correcto.
<code>php artisan route:list</code>	Correcto	Las rutas cargan, pero se detectaron rutas publicas de utilidad/prueba.
<code>php artisan test</code>	2 passed	Solo son tests de ejemplo; no cubren negocio real.
Lint PHP sin <code>vendor</code>	Sin errores de sintaxis	La salida se ensucia por el warning de <code>pdo_oci</code> .
<code>php artisan migrate:status</code>	Base responde	Hay 3 migraciones pendientes.
<code>composer audit --locked</code>	Falla por advisories	22 vulnerabilidades en 11 paquetes y 3 paquetes abandonados.
<code>npm.cmd audit --audit-level=moderate</code>	Falla por advisories	224 vulnerabilidades: 78 low, 88 moderate, 46 high, 12 critical.

<code>php artisan db:seed --class=RolesTableSeeder</code>	Falla	Class "App\Models\Rol" not found.
---	-------	-----------------------------------

Fallas y Riesgos Detectados

1. Dependencias vulnerables y obsoletas

- `composer audit` encontro 22 advisories. Destacan vulnerabilidades criticas/altas en `phpoffice/phpspreadsheet`, `laravel/framework`, `symfony/*`, `phpunit`, `guzzlehttp/psr7` y otros.
- `npm audit` encontro 224 vulnerabilidades, incluyendo 12 criticas. Paquetes principales del arbol: `laravel-mix`, `axios`, `webpack`, `lodash`, `postcss`, `sha.js`, `pbkdf2`, `node-forge`.
- Hay paquetes abandonados en Composer: `fruitcake/laravel-cors`, `swiftmailer/swiftmailer` y `fzaninotto/faker`.

2. Base de datos desalineada con el codigo

- `php artisan migrate:status` muestra pendientes: `add_location_id_to_employees_table`, `add_adicional_to_sales_table` y `add_vehicle_plate_to_sales_table`.
- El codigo ya usa `sales.adicional` y `sales.vehicle_plate` en controladores y vistas. Referencias: `app/Http/Controllers/SaleController.php:427`, `:430`, `:493`, `:494`, `resources/views/sales/historico.blade.php:176`.
- Si esas migraciones no se ejecutan, se esperan errores SQL de columna inexistente en registro e historial de ventas/recalibracion.

3. Seeders rotos y datos iniciales no reproducibles

- `database/seeders/RolesTableSeeder.php:6` usa `App\Models\Rol`, pero el modelo existente es `App\Models\Role`.
- `database/seeders/UsuariosTableSeeder.php:6` usa `App\Models\Usuario`, que no existe.
- Varios seeders usan columnas en espanol que no coinciden con los modelos actuales: `nombre`, `rol_id`, `activo`, `razon_social`, `dni_ruc`, etc.
- Esto impide levantar un ambiente limpio con `migrate --seed` y obliga a depender de dumps SQL.

4. Rutas publicas que deberian estar protegidas o eliminadas

- `routes/web.php:47` expone `/storage` y ejecuta `Artisan::call('storage:link')` sin autenticacion. Es una ruta operativa que no deberia estar disponible publicamente.
- `routes/web.php:56` expone `/sales/prueba` fuera del grupo `auth`. Aunque sea prueba, deja una vista accesible sin sesion.
- El grupo protegido empieza recién en `routes/web.php:61`.

5. Autorizacion dispersa

- La mayoría de rutas protegidas usan solo `auth`, no middleware por rol ni policies.

- El rol se compara manualmente en muchos controladores con `auth()->user()->role->nombre`. Si una acción olvida ese filtro, un usuario autenticado podría entrar por URL directa.
- La lógica de sedes también está repetida en controladores; conviene centralizarla para evitar fugas de datos entre sedes.

6. Actualizaciones masivas con `$request->all()`

- Se detectaron actualizaciones directas con datos completos del request: `ClientController.php:117`, `OrderController.php:218`, `PurchaseController.php:218`, `SedeController.php:85`.
- Aunque haya validación previa, `$request->all()` puede incluir campos extra. Lo correcto es usar `$request->validated()` o `$request->only([...])`.

7. Configuración local peligrosa si pasa a producción

- `.env:2` tiene `APP_ENV=local` y `.env:4` tiene `APP_DEBUG=true`.
- En producción eso mostraría trazas, paths, consultas y detalles internos al usuario.
- `.env.example` mantiene valores genéricos y `DB_DATABASE=laravel`, distinto del entorno local `ava`.

8. Warning permanente de PHP por extensión faltante

- Cada comando PHP muestra: no se puede cargar `pdo_oci / php_pdo_oci.dll`.
- No bloquea MySQL, pero ensucia pruebas, logs y despliegues. Si no se usa Oracle, se debe deshabilitar esa extensión en `php.ini`.

9. Pruebas insuficientes

- `tests/Feature/ExampleTest.php` solo valida que `GET /` devuelva 200.
- `tests/Unit/ExampleTest.php` solo hace `assertTrue(true)`.
- `phpunit.xml:25` tiene comentada la base en memoria, así que no hay entorno de test aislado.
- No existen pruebas de login real, permisos por rol, ventas, compras, cierres, créditos/contratos, importación Excel, stock, reportes o restricciones por sede.

10. Duplicidad y mantenimiento de rutas

- El mismo controlador aparece con dos recursos para cierre: `routes/web.php:191` y `routes/web.php:236`.
- Hay endpoints AJAX bajo rutas web con prefijo visual `/api`, lo que puede confundir seguridad, versionado y consumo externo.

11. Dumps SQL dentro del repositorio

- Existen `ava_db.sql` y `ava-2.sql` en la raíz.

- Los dumps incluyen estructura y datos de usuarios con hashes. Si contienen datos reales, representan riesgo de privacidad y de malas practicas de despliegue.

Revision de Experiencia de Usuario

- La pantalla de ingreso esta construida como vista partida: formulario a la izquierda, imagen a la derecha, logos de proveedor debajo y boton central.
- En escritorio deberia verse limpia y reconocible; en movil la imagen derecha se oculta por clases Bootstrap.
- El formulario usa label "Usuario" aunque el campo internamente se llama `email`; esto funciona, pero puede confundir si el sistema acepta usuario y no correo.
- Solo se muestra error bajo el campo de usuario; no hay feedback especifico bajo password.
- No se observa enlace de recuperacion de contraseña ni estado de carga al enviar.
- El boton de ver/ocultar password existe y mejora usabilidad.
- El servidor local arranco correctamente en primer plano, pero no se pudo mantener vivo en segundo plano desde el sandbox para captura HTTP completa. Esto se reporta como limitacion del entorno de prueba, no como falla confirmada del sistema.

Buenas Practicas Observadas

- CSRF esta activo y no hay excepciones globales en `VerifyCsrfToken`.
- El login regenera sesion al autenticar y aplica limitador de intentos.
- Muchos controladores aplican filtros por sede, lo cual apunta a una separacion de datos necesaria para la operacion.
- Las rutas principales estan agrupadas bajo `auth`.
- Composer valida correctamente y no se detectaron errores de sintaxis PHP en la pasada de lint.

Prioridad Recomendada

Prioridad	Accion	Motivo
Alta	Actualizar dependencias Composer y npm con pruebas de regresion.	Hay vulnerabilidades criticas y altas.
Alta	Ejecutar o corregir migraciones pendientes en ambiente controlado.	El codigo usa columnas que la base actual aun no tiene.
Alta	Eliminar/proteger <code>/storage</code> y vistas de prueba publicas.	Reducen superficie de ataque y exposicion accidental.
Alta	Arreglar seeders y alinear nombres de columnas/modelos.	Permite instalar el sistema sin depender de dumps manuales.
Media	Crear middleware/policias por rol y sede.	Evita accesos por URL directa y reduce duplicacion.
Media	Reemplazar <code>\$request->all()</code> por datos validados.	Evita asignaciones no intencionales.
Media	Configurar ambiente de test aislado y cubrir flujos criticos.	Los tests actuales no protegen negocio.

Baja	Limpiar warning pdo_oci y revisar copy/feedback del login.	Mejora mantenibilidad y experiencia de usuario.
------	--	---

Conclusion

El sistema tiene base funcional y una organizacion de modulos entendible, pero aun no esta en un estado confiable para operar sin riesgos: necesita alinear base de datos, limpiar rutas publicas, actualizar dependencias, recuperar seeders y crear pruebas reales. La mayor prioridad tecnica es estabilizar seguridad y esquema; la mayor prioridad funcional es cubrir ventas, pagos, stock y permisos con pruebas automatizadas.

Notas: esta revision no modifica logica del sistema. Los scripts temporales usados para generar este PDF pueden eliminarse despues de producir el archivo.